Ejercicio 1: Pila enlazada de enteros

Prompt redactado:

Estructura solicitada: Implementación de una Pila de enteros (PilaEnlazadaEnteros) mediante una Lista Enlazada Simple.

Explicación del tope e inserción/eliminación $O(1)$:

- Explica que el nodo head representa directamente el tope de la pila.
- Al insertar (push), el nuevo nodo se coloca al inicio haciendo que apunte al head actual, convirtiéndose en el nuevo head.
- Al extraer (pop), simplemente se desplaza el head al nodo siguiente (head = head.siguiente).
- Dado que ambas operaciones ocurren exclusivamente en la cabeza de la lista, no requieren recorrer la estructura, garantizando una complejidad de $O(1)$ constante.

Operaciones a implementar:

1. push(int x) (apilar)
2. pop() (desapilar)
3. peek() (consultar tope)
4. isEmpty() (verificar si está vacía)
5. contiene(int x) (buscar valor)
6. imprimir() (mostrar contenido)

Casos especiales: Intentar desapilar o consultar el tope en una pila vacía (lanzar excepción o mostrar aviso).

Código y Pruebas: Métodos comentados y prueba en main mostrando inserción, consulta de tope, búsqueda y desapilado.

Ejercicio 2: Cola enlazada de enteros

Prompt redactado:

Estructura solicitada: Implementación de una Cola de enteros (ColaEnlazadaEnteros) mediante una Lista Enlazada Simple.

Explicación del uso de head y tail:

- Explica que la cola requiere dos punteros/referencias: head (apunta al primer elemento ingresado, por donde se desencola) y tail (apunta al último elemento ingresado, por donde se encola).

- Justifica que si solo se utilizara head, encolar al final requeriría recorrer toda la lista desde el principio hasta el último nodo en cada inserción, degradando la complejidad de $O(1)$ a $O(n)$.

Operaciones a implementar:

1. enqueue(int x) (encolar al final con tail en $O(1)$)
2. dequeue() (desencolar al inicio desde head en $O(1)$)
3. front() (consultar frente)
4. isEmpty() (verificar vacía)
5. contiene(int x) (buscar valor)
6. imprimir() (mostrar contenido)

Casos especiales: Al desencolar el último elemento de la lista, actualizar tanto head como tail a null.

Código y Pruebas: Método main probando encolar varios elementos, desencolar y verificar estado.

Ejercicio 3: Pila genérica

Prompt redactado:

Estructura solicitada: Convertir la Pila enlazada en una clase genérica PilaEnlazada<T>.

Justificación teórica de Generics:

- Explica qué significa el uso de genéricos en Java (<T>) y por qué la lógica de manipulación de la pila (LIFO) no depende del tipo de dato almacenado: los punteros de los nodos se enlazan de la misma manera independientemente de si la carga útil es un entero, un texto o una clase personalizada.

Operaciones a implementar: push(T elemento), pop(), peek(), isEmpty(), contiene(T elemento) e imprimir().

Código y Pruebas: Probar instanciando PilaEnlazada<Integer>, PilaEnlazada<String> y una pila con objetos de una clase propia (ej. PilaEnlazada<Producto>).

Ejercicio 4: Cola genérica

Prompt redactado:

Estructura solicitada: Convertir la Cola enlazada en una clase genérica ColaEnlazada<T>.

Justificación teórica: Explica en los comentarios cómo la política FIFO (First In, First Out) se preserva intacta sin importar el tipo de dato, ya que la secuencia temporal de inserción con tail y extracción con head opera exclusivamente sobre las referencias de la envoltura Nodo<T>.

Operaciones: enqueue(T elemento), dequeue(), front(), isEmpty() e imprimir().

Código y Pruebas: Instanciar y probar el comportamiento con una cola de nombres ColaEnlazada<String> y una cola de objetos de tipo Cliente (ColaEnlazada<Cliente>).

Ejercicio 5: Historial de navegación con pila

Prompt redactado:

Estructura solicitada: Sistema simulador de historial de navegación web en Java utilizando la estructura PilaEnlazada<String>.

Justificación LIFO: Explica por qué una Pila representa fielmente la acción de "volver atrás" en un navegador: la página web más reciente que se ha cargado es la primera a la que regresamos cuando se invoca la operación retroceder (pop).

Operaciones:

1. visitarPagina(String url): Apila una nueva dirección web (push).
2. volverAtras(): Desapila la página actual para regresar a la previa (pop).
3. consultarPaginaActual(): Devuelve el sitio posicionado en el tope (peek).
4. imprimirHistorial(): Muestra las páginas apiladas.

Casos especiales: Intentar volverAtras() cuando el historial no tiene páginas web apiladas.

Código y Pruebas: Simular la visita a 3 sitios y realizar 2 retrocesos mostrando los mensajes correspondientes.

Ejercicio 6: Cola de atención de clientes

Prompt redactado:

Estructura solicitada: Sistema de atención al cliente para un banco utilizando una ColaEnlazada<Cliente>.

Modelado de datos: Clase Cliente con los atributos nombre (String), numeroTurno (int) y motivoConsulta (String).

Justificación FIFO: Explica por qué la estructura idónea es una Cola: representa de forma equitativa y cronológica el orden de llegada de las personas al banco, asegurando que el primer cliente en llegar sea el primero en ser atendido.

Operaciones:

1. agregarCliente(Cliente c): Encola al final de la fila (enqueue).
2. atenderProximo(): Desencola al cliente del frente y muestra sus datos (dequeue).
3. consultarProximo(): Muestra quién es el siguiente cliente a ser atendido sin quitarlo (front).
4. imprimirFila(): Muestra la lista de clientes en espera.

Código y Pruebas: Encolar 3 clientes y atender a 2 de ellos en el orden correcto.

Ejercicio 7: Verificador de paréntesis balanceados

Prompt redactado:

Estructura solicitada: Programa en Java para validar expresiones matemáticas balanceadas utilizando una PilaEnlazada<Character>.

Lógica con Pila:

- Recorrer la cadena carácter por carácter.
- Si se encuentra un paréntesis de apertura '(', apilarlo (push).
- Si se encuentra un paréntesis de cierre ')', verificar que la pila no esté vacía y desapilar (pop).

Casos especiales de invalidez:

1. Intentar desapilar cuando se encuentra un ')' y la pila está vacía.
2. Terminar de recorrer la cadena y comprobar que la pila no haya quedado vacía (quedaron paréntesis abiertos).

Código y Pruebas: Validar los ejemplos válidos "(2 + 3) * (5 - 1)" y "((a + b) * c)", y los ejemplos inválidos "(2 + 3" y "())(".

Ejercicio 8: Invertir una palabra con pila

Prompt redactado:

Estructura solicitada: Programa que invierta una cadena de texto (String) utilizando una PilaEnlazada<Character>.

Justificación LIFO: Explica en el prompt por qué la política LIFO permite invertir secuencias de caracteres: al apilar los caracteres en orden de izquierda a derecha, el último carácter insertado queda ubicado en la cima de la pila, provocando que al desapilarlos sucesivamente salgan en orden inverso (de derecha a izquierda).

Algoritmo:

1. Recorrer la palabra e insertar cada carácter en la pila con push.
2. Desapilar secuencialmente con pop construyendo la palabra invertida hasta que la pila quede vacía.

Código y Pruebas: Probar invirtiendo la palabra "algoritmo" para obtener "omtirogla".

Ejercicio 9: Cola de impresión

Prompt redactado:

Estructura solicitada: Sistema de gestión de cola de impresión con una ColaEnlazada<TrabajoImpresion>.

Modelado de datos: Clase TrabajoImpresion con nombreArchivo (String), cantidadPaginas (int) y usuario (String).

Justificación FIFO: Explica por qué un spooler de impresión debe procesar los trabajos estrictamente con la disciplina FIFO (First In, First Out): previene que solicitudes recientes desplacen trabajos enviados previamente, garantizando la equidad del servicio.

Operaciones:

1. agregarTrabajo(TrabajoImpresion trabajo) (enqueue)
2. imprimirProximoTrabajo() (dequeue)
3. consultarProximoArchivo() (front)
4. mostrarTrabajosPendientes()

Código y Pruebas: Cargar 3 archivos a la cola de impresión y procesar el flujo.

Ejercicio 10: Lista doblemente enlazada genérica

Prompt redactado:

Estructura solicitada: Clase genérica ListaDobleEnlazada<T>.

Explicación de nodos y referencias:

- Explicar la diferencia con la lista simple: cada NodoDoble<T> contiene tres atributos: el dato tipo T, la referencia al siguiente nodo y la referencia al nodo anterior.
- Explicar que la lista mantiene dos punteros principales: head (cabeza) y tail (cola).

Operaciones a implementar:

1. insertarAlInicio(T dato): Actualiza referencias siguiente y anterior.

2. insertarAlFinal(T dato): Conecta con tail y reconecta en ambos sentidos.
3. eliminar(T dato): Busca el nodo y "saltea" sus conexiones actualizando nodo.anterior.siguiente = nodo.siguiente y nodo.siguiente.anterior = nodo.anterior.
4. imprimirHaciaAdelante(): Recorre desde head hacia tail.
5. imprimirHaciaAtras(): Recorre desde tail hacia head usando los punteros anterior.

Casos especiales al eliminar:

- Eliminar el único nodo existente (head y tail pasan a ser null).
- Eliminar el primer nodo (actualizar head y hacer head.anterior = null).
- Eliminar el último nodo (actualizar tail y hacer tail.siguiente = null).

Código y Pruebas: Método main demostrando inserciones, eliminaciones e impresiones en ambas direcciones.